Kaleido

What it is. What it solves. How it works.

YOUR READY-TO-USE DESCRIPTION

Kaleido is a .NET framework for building business services as reusable queries and executable steps. It runs your business code, tracks process progress, and returns what can happen next, through application APIs and AI tools.

Suggested pace: about 18-21 minutes, plus questions. Rehearse from the Say sections; present from the quick cues. The PowerPoint contains the same talk track in its speaker notes.

THE 14-SLIDE CUE SHEET

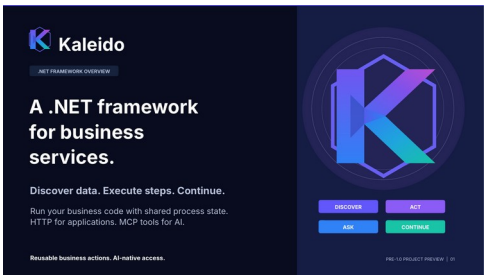
#	FOCUS	REMEMBER
01	What it is	A .NET framework that runs business steps.
02	The three parts	Queryable reads. Process acts. Registry describes.
03	The problem	Stop rebuilding process logic in each consumer.
04	Why I started	Make the step concept tangible.
05	The step concept	An action, not a screen or a fixed sequence.
06	Build time	Declare, implement, register, expose.
07	Run time	Load, check, run, save, return next.
08	Where it runs	Different interfaces; the same business execution.
09	The AI mechanism	Metadata becomes tools; calls reach handlers.
10	A worked action	Search reads. Selection acts. State guides next.
11	Continuity	Reload process state, not only chat history.
12	Platform fit	A process runtime, not the entire case platform.
13	Authority and preview	Mechanisms are not proof of permission or readiness.
14	The close	Define. Discover. Execute. Continue.

Each slide starts on the next page in order. Q&A follows on pages 16-17.

Basis: the supplied v3 deck and guide, the available source export, and the review reports. Examples are illustrative; no runtime or release validation was performed for this update.

SLIDE 01 / 45-60 SECONDS

A .NET framework for business services.



REMEMBER

Name the category first: a .NET framework that runs business steps, not just a catalog.

On screen: Start with the framework definition. Then point to discover, act, ask, and continue.

SAY

Kaleido is a .NET framework for building business services. It lets us define the information a business can provide and the actions it can perform, then make those operations usable by applications and AI assistants.

It does more than publish a list of operations. It runs our business handlers, tracks process progress, and returns what can happen next. So the website or the assistant does not have to invent a separate process model.

Think of an action like selecting a provider or submitting a request. We write the business behavior in the service. Kaleido supplies the common mechanisms around that behavior, including the process state and the interface an AI can use.

This is a pre-1.0 preview. I will explain what it is, the problem it addresses, and then walk through how a step actually works.

TRANSITION TO THE NEXT SLIDE

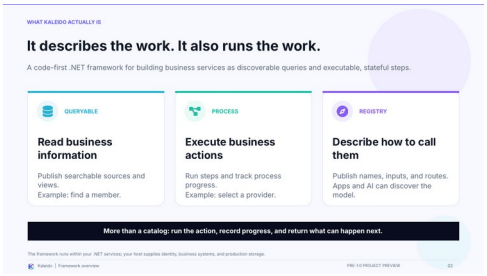
Let me separate the three parts of the framework before we look at the problem.

Presenter reminder (not spoken): AI-native refers to the implemented MCP adapter. Kaleido is not a built-in language model or a completed production application.

Basis: v4 slide 01. Underlying source paths and scope are recorded in the PowerPoint speaker notes.

SLIDE 02 / 60-90 SECONDS

It describes the work. It also runs the work.



REMEMBER

Queryable reads. Process acts. Registry describes how to call them.

On screen: Point to each card in order. Finish on the dark strip: more than a catalog.

SAY

There are three things working together here. Queryable exposes business information: for example, searching for a member or finding provider locations. Process executes business actions: for example, recording the provider selected for a request. Registry describes those operations so a consumer can discover their names, inputs, and how to call them.

When I use the word capability, I simply mean one of those things the business can provide or do. It is not a separate product we have to buy or a new kind of business rule.

The important distinction is that Kaleido includes execution. It is not only documentation for an API. For a process step, the framework checks the declared rules, invokes our handler, maintains process state, and returns guidance about the next work.

Developers use it inside their .NET services. We still supply the business code, connect the real systems, configure identity and access, and choose production storage. HTTP exposes the model to applications; the MCP adapter exposes it as tools to an AI assistant.

TRANSITION TO THE NEXT SLIDE

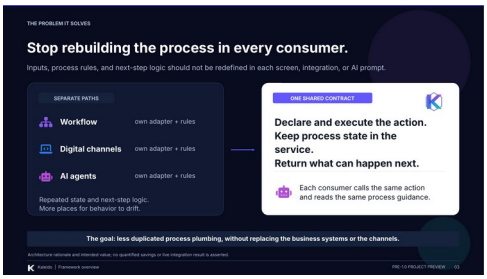
The reason to put those mechanisms in one framework is the duplication they can otherwise create.

Presenter reminder (not spoken): The three cards describe complementary roles, not three separately deployed products. Registry discovery does not replace execution-time process or authorization checks.

Basis: v4 slide 02. Underlying source paths and scope are recorded in the PowerPoint speaker notes.

SLIDE 03 / 60-75 SECONDS

Stop rebuilding the process in every consumer.



REMEMBER

The target is duplicated process plumbing, not the removal of all integration work.

On screen: Contrast the three paths on the left with shared execution and state on the right.

SAY

The problem I am trying to solve is not simply that we need another API. It is that every new consumer can become another place where we rebuild the same understanding of the business.

The website has to know which inputs to send, which action is valid, and what to show next. A workflow integration needs similar knowledge. Then an AI assistant arrives, and we risk rebuilding those assumptions in another adapter or prompt.

Kaleido gives those consumers a common place to obtain that information and perform the work. The service owns the action and its process state. The consumer calls the action and reads the returned guidance instead of maintaining a separate version of the journey.

We could build these mechanisms ourselves without Kaleido. The framework is intended to make them reusable and consistent across services. It does not eliminate domain code, user-interface design, or boundary mapping, and I am not claiming a measured cost saving yet.

TRANSITION TO THE NEXT SLIDE

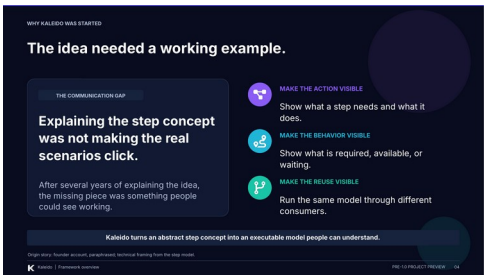
That same communication problem is also part of why I started the project.

Presenter reminder (not spoken): This is the architectural value proposition, not a claim that all existing systems duplicate logic or that Kaleido is the only way to avoid duplication.

Basis: v4 slide 03. Underlying source paths and scope are recorded in the PowerPoint speaker notes.

SLIDE 04 / 60-75 SECONDS

The idea needed a working example.



REMEMBER

I built Kaleido to make the step concept tangible, not just explain it again.

On screen: Speak personally. Pause after the communication gap, then move through action, behavior, and reuse.

SAY

For the past few years, I have been trying to explain this idea of steps. I could describe the concept, but when we got into specific scenarios, I was not making it clear how the pieces would actually work together.

What I wanted people to see was the behavior: what an action needs, what happens when it runs, what becomes available afterward, and what happens when the process needs more information.

That is the main reason I started Kaleido. Instead of asking people to imagine the model from another explanation, I wanted an executable example we could run, inspect, and discuss. The point was to make the idea concrete.

The AI importance has grown from that foundation. Once the actions, inputs, and ways to continue are explicit, an AI assistant has something structured to work with too.

TRANSITION TO THE NEXT SLIDE

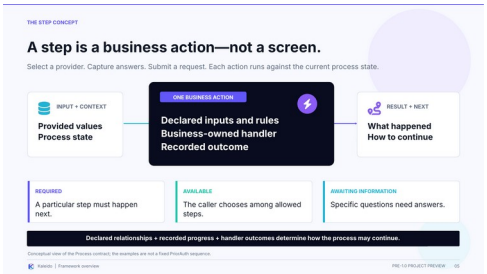
Here is the distinction that makes the whole model easier to understand: a step is an action, not a screen.

Presenter reminder (not spoken): First-person wording is a paraphrase of your account. Do not attribute it to a repository quotation or imply that AI was the original motivation.

Basis: v4 slide 04. Underlying source paths and scope are recorded in the PowerPoint speaker notes.

SLIDE 05 / 90-120 SECONDS

A step is a business action - not a screen.



REMEMBER

Inputs + current progress go into an action; an outcome + next work come back.

On screen: Trace left to right. Then use the three continuation cards to explain that the process is not a page sequence.

SAY

Take selecting a provider. A website might show a list. A contact-center employee might help someone make the selection. An AI assistant might search and present the matching records. Those are different interactions around the same business action.

The step is that business action. It has declared inputs, relationships to other steps, and a handler that implements what it means. It is not the screen, the button, or the chat message used to request it.

The order comes from the declared relationships, the recorded progress, and any next direction returned by the handler. As an illustration, imagine that adding contact details and selecting a provider are both available after intake. Either could happen first if the rules allow it, while a review step could depend on both being complete.

That is how flexibility and order coexist. Sometimes one particular step is required next. Sometimes there are several allowed choices. Sometimes the process needs answers to specific questions. The runtime checks those constraints; the interface presents the work.

The framework does not decide which provider is best or whether a request deserves approval. That business meaning stays in the application and with the people or systems authorized to decide it.

TRANSITION TO THE NEXT SLIDE

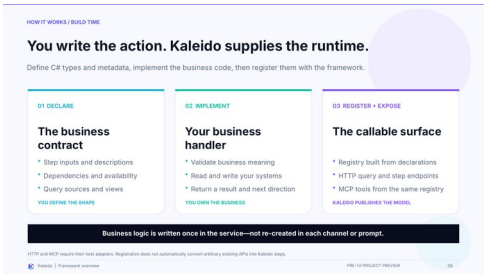
Now let me show what a developer actually writes to create one of those steps.

Presenter reminder (not spoken): The contact-details example is hypothetical and illustrates the relationship model; it is not a claim about the exact PriorAuth sequence. The three cards are not an exhaustive state list.

Basis: v4 slide 05. Underlying source paths and scope are recorded in the PowerPoint speaker notes.

SLIDE 06 / 90-120 SECONDS

You write the action. Kaleido supplies the runtime.



REMEMBER

Declare the contract, implement the handler, then register and expose the model.

On screen: Walk across the three columns. Clearly distinguish what the developer writes from what the framework derives.

SAY

This is the build-time view. First, we declare the operation in C#: its input type, a meaningful name and description, and the process relationships that apply. We also define the query sources or views needed to find information. That is what code-first means here.

Second, we implement the business handler. For provider selection, that code checks the business meaning of the selection, reads or writes the appropriate systems, and returns an outcome. The business code can also direct the process to a required next step or request more information.

Third, we register the relevant assemblies with Kaleido and configure the host adapters we need. The framework builds registries from those declarations. The HTTP layer exposes the query and step operations. The MCP adapter translates the registry into tools with input schemas and descriptions.

The important point is what we do not have to define independently: a business action for the website and a second business action for the AI. Both can reach the same implementation.

This is not automatic conversion of an arbitrary existing API. Developers adopt the contracts, supply the handlers or query implementations, and configure the host. Kaleido supplies the reusable execution and discovery machinery around that work.

TRANSITION TO THE NEXT SLIDE

That is how we build the action. Here is what happens when somebody calls it.

Presenter reminder (not spoken): No executable code snippet is shown. The source export contains stale internal names, so this explanation intentionally uses the documented public step/handler/registration concepts.

Basis: v4 slide 06. Underlying source paths and scope are recorded in the PowerPoint speaker notes.

SLIDE 07 / 90-120 SECONDS

What happens when someone calls a step?



REMEMBER

Load state, check the request, run the handler, record the outcome, return next work.

On screen: Start on the request bar. Move through all five boxes, then land on the three kinds of continuation data.

SAY

Imagine the caller requests our provider-selection action. It sends the selected values and, when continuing existing work, the process ID. The host must establish the appropriate identity and authorization; the diagram focuses on the process execution after that boundary.

First, Kaleido loads the current process state, or creates a process when an initial step starts new work. Second, it checks the request: does the input match the declared structure, have required earlier steps happened, and is this step currently permitted?

Third, the runtime invokes our business handler. This is where our code checks that the selected provider location belongs to the provider and is valid for this request. Framework validation and business validation are different responsibilities.

Fourth, on the normal completion path, Kaleido records the outcome and saves the updated framework process state. Fifth, it returns the result, the process identity, and how to continue.

That last part is the key. The response is more than a success code. It can identify required work, available choices, or questions that need answers. The next interaction follows that information rather than a hardcoded page number.

If the request is not valid or not available, it does not simply follow this happy path. The framework reports the relevant failure or process condition instead.

TRANSITION TO THE NEXT SLIDE

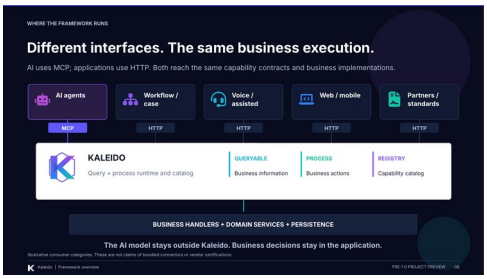
The same execution mechanism sits underneath all of these interfaces.

Presenter reminder (not spoken): This is a conceptual happy path, not a live trace. Arbitrary business side effects and framework-state saves are not automatically one transaction; retry/completion validation remains pre-1.0 work.

Basis: v4 slide 07. Underlying source paths and scope are recorded in the PowerPoint speaker notes.

SLIDE 08 / 60-75 SECONDS

Different interfaces. The same business execution.



REMEMBER

The interface changes; the framework and business handler underneath do not.

On screen: Follow the AI lane through MCP, then the HTTP lanes. Point to the runtime and then the business implementations.

SAY

This picture puts the framework in context. An AI assistant comes through the MCP adapter. Web applications, workflow integrations, assisted channels, and partner adapters can come through HTTP. They reach the same query and process model.

The middle is not just a passive catalog. Kaleido includes the runtime we just walked through, along with the registries and information contracts. At the bottom are the business handlers, domain services, and persistence that implement the actual business.

The language model stays outside Kaleido. The application still owns the business meaning. An assistant can help someone interact with a process without becoming the place where the rules and state live.

These are categories of consumers, not a claim that every connector shown is packaged with the project. The important architectural choice is to keep execution independent of presentation.

TRANSITION TO THE NEXT SLIDE

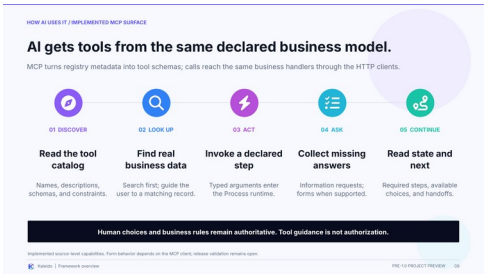
Now the AI story should be easier to explain, because it uses the same mechanism rather than introducing another one.

Presenter reminder (not spoken): The drawing omits detailed deployment and credential topology. It is not evidence of completed downstream delegation or delivered vendor integrations.

Basis: v4 slide 08. Underlying source paths and scope are recorded in the PowerPoint speaker notes.

SLIDE 09 / 90-105 SECONDS

AI gets tools from the same declared business model.



REMEMBER

Metadata becomes tools; tool calls reach business handlers; returned state guides continuation.

On screen: Point to the subtitle for the mechanism, then walk through discover, look up, act, ask, and continue.

SAY

The MCP adapter turns the caller-visible registry into tools. It uses the names, descriptions, input schemas, and constraints we declared for the business operations. Tool calls then go through the Kaleido HTTP clients to the business services.

A lookup tells the interaction where an input value comes from. Rather than guessing an internal identifier, the assistant can search for a real record and let the user choose. Where the client supports forms, the adapter can guide that selection directly.

When the assistant invokes a step, it reaches the runtime and the business handler we already discussed. If information is missing, the service can return specific questions. The model is guided to obtain the user's answers, not invent the business facts.

Afterward, the returned state says what is required or available. The default guidance tells the assistant to offer available choices to the user and follow that process information.

That is the AI importance here: the business publishes a usable interface for action, not just text for the assistant to read. But instructions and tool hints remain guidance; they do not replace server-side validation or authorization.

TRANSITION TO THE NEXT SLIDE

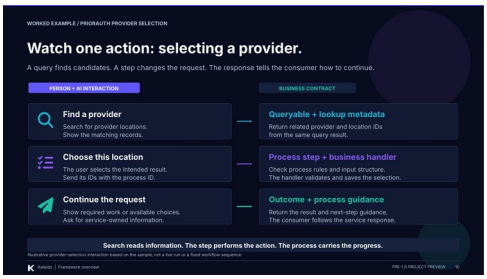
Let us follow one small example all the way through, without changing the action halfway through the explanation.

Presenter reminder (not spoken): Form behavior depends on the MCP client. AI-native does not imply built-in inference, autonomous planning, or guaranteed model compliance.

Basis: v4 slide 09. Underlying source paths and scope are recorded in the PowerPoint speaker notes.

SLIDE 10 / 90-105 SECONDS

Watch one action: selecting a provider.



REMEMBER

Search reads. Selection acts. Returned state tells us how to continue.

On screen: For each row, say the person/AI interaction on the left and immediately explain the mechanism on the right.

SAY

This is an illustrative interaction based on the provider-selection pattern in the sample, not a recording of a live run.

Person: "Help me find the provider location for this request." The assistant searches and shows matching records. Underneath, that is a query. It reads information; it has not yet changed the provider selection on the request.

Person: "Use this location." Now there is a business action to perform. The consumer takes the related provider and location identifiers from the selected result and calls the step with the process ID.

Kaleido checks the process constraints and the input structure. The business handler checks the relationship and whether the selection is valid for the request, then performs the action. Merely displaying a valid-looking choice in a form is not a substitute for those server-side checks.

The consumer then reads the outcome and next-step guidance. It might need to present a required action, offer allowed choices, or collect service-owned information. It should use the actual response, not assume which screen comes next.

A website could perform the same search, selection, and continuation with different controls. The business action stays in the service; the conversation is only one interface to it.

TRANSITION TO THE NEXT SLIDE

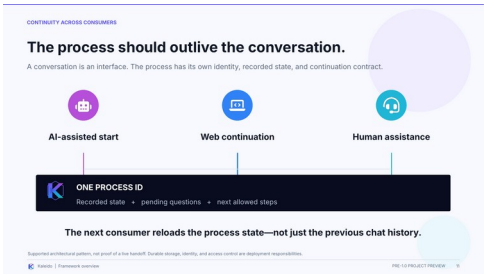
Because the progress belongs to the process, it can also survive a change of interface.

Presenter reminder (not spoken): The dialogue and labels are explanatory, not literal tool names, API payloads, or a fixed PriorAuth workflow. No actual service-owned question or attestation is invented.

Basis: v4 slide 10. Underlying source paths and scope are recorded in the PowerPoint speaker notes.

SLIDE 11 / 60-75 SECONDS

The process should outlive the conversation.



REMEMBER

The next authorized consumer reloads process state, not just chat history.

On screen: Trace the three consumers, then point to the one-process-ID bar underneath.

SAY

A conversation is one way to interact with the business. It should not be the only place where the business knows what has happened.

The process has its own identity and current recorded state, including pending information and next steps. A subsequent authorized consumer can use that identity to read the state before continuing.

The picture shows an AI-assisted start, a web continuation, and human assistance. The important idea is not those particular channels. It is that changing the interface should not require reconstructing the business process from a transcript.

That pattern depends on durable storage and appropriate access control in the deployment. The process ID identifies the work; it does not by itself prove that the person has permission to access that case. It is also current process state, not a claim that every historical business fact is stored there.

TRANSITION TO THE NEXT SLIDE

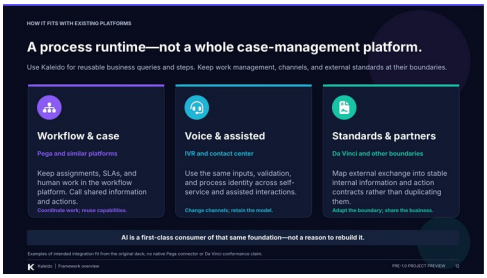
This is why I see Kaleido working alongside existing platforms, rather than replacing everything around it.

Presenter reminder (not spoken): Cross-channel continuation is distinct from cross-processor handoff. The sample documents open handoff validation work. Do not claim every transfer path has been proven.

Basis: v4 slide 11. Underlying source paths and scope are recorded in the PowerPoint speaker notes.

SLIDE 12 / 60-75 SECONDS

A process runtime - not a whole case-management platform.



REMEMBER

Be precise: Kaleido executes business processes, but is not the entire work-management environment.

On screen: Start with the title distinction, then briefly cover the three platform boundaries.

SAY

There is a wording trap I want to avoid. Saying Kaleido is not an engine can make it sound like it is only an API description. It does include a process runtime: it checks step rules, runs handlers, and records progress.

What this presentation does not position it as is a replacement for an entire workflow or case-management platform. That platform can retain its assignments, human tasks, and service-level management while calling reusable business information and actions.

Voice, web, and assisted channels retain their presentation and interaction responsibilities. Standards and partner adapters retain their external exchange boundary. They can map into the business operations rather than define those operations again.

The AI assistant is another first-class consumer through the implemented MCP adapter. The categories here describe integration fit, not bundled connectors or a standards certification.

TRANSITION TO THE NEXT SLIDE

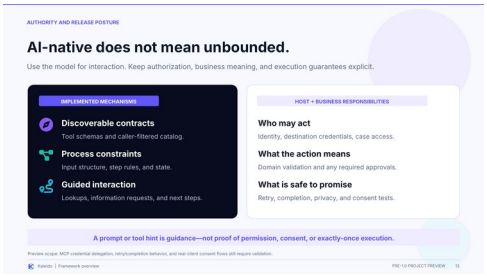
The remaining boundary is authority: who can act, what the action means, and what the preview is ready to promise.

Presenter reminder (not spoken): Do not describe Kaleido as having no process execution, or imply that Pega, IVR, or Da Vinci integrations are delivered and certified.

Basis: v4 slide 12. Underlying source paths and scope are recorded in the PowerPoint speaker notes.

SLIDE 13 / 75-90 SECONDS

AI-native does not mean unbounded.



REMEMBER

The framework provides mechanisms; the host and business retain authority and release responsibilities.

On screen: Separate implemented mechanisms on the left from deployment and business responsibilities on the right.

SAY

The left side lists implemented mechanisms: tool schemas, caller-filtered discovery, input validation, process rules, lookups, and continuation guidance. Those mechanisms make the interaction more explicit.

They do not transfer authority to the model. The host and application must establish who may act, which records they may access, what an action means, and whether an approval or confirmed consent is required.

A prompt that says "ask for confirmation" is useful guidance. It is not proof that a person consented. And storing a process outcome does not automatically guarantee that every external side effect happens exactly once.

That is also why the deck remains a pre-1.0 preview. The reviews identify unresolved work around MCP credential delegation, retries and completion, and real-client consent behavior. Those need to be resolved and tested against the actual release candidate.

I am presenting an implemented framework and an AI interface, not claiming that every production guarantee has already been established.

TRANSITION TO THE NEXT SLIDE

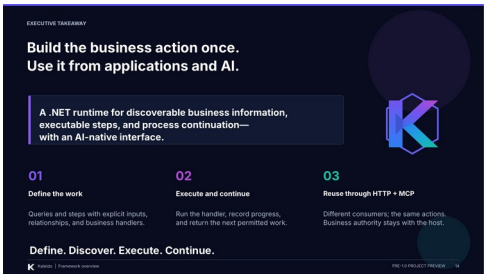
So the takeaway is a concrete framework, a repeatable execution model, and a shared interface for applications and AI.

Presenter reminder (not spoken): No cleanup recommendation is treated as implemented. Do not promise prompt-injection immunity, automatic business approval, or arbitrary exactly-once external effects.

Basis: v4 slide 13. Underlying source paths and scope are recorded in the PowerPoint speaker notes.

SLIDE 14 / 45-60 SECONDS

Build the business action once. Use it from applications and AI.



REMEMBER

Define the work. Execute and continue. Reuse through HTTP and MCP.

On screen: Return to the concrete definition, then finish on the four verbs along the bottom.

SAY

If someone asks what Kaleido is, this is the answer I want them to leave with: it is a .NET framework for building business services as discoverable queries and executable steps, with process state and guidance about what comes next.

We define the action and its business handler in the service. Kaleido runs it within the declared process rules, records progress, and exposes the model to applications through HTTP and to AI assistants through MCP.

The problem it addresses is having to rebuild that same process plumbing and understanding for every new consumer.

I started the project to make the step concept something people could see working. The AI opportunity makes that explicit model even more useful, without moving business authority into a chatbot.

Define the work. Discover it. Execute the action. Continue the process.

FINISH

Pause, then invite the audience to describe a concrete process they would like to walk through.

Presenter reminder (not spoken): Close on the supported mechanism and intended value, not an unmeasured saving or a 1.0 release commitment.

Basis: v4 slide 14. Underlying source paths and scope are recorded in the PowerPoint speaker notes.

Q&A / SUGGESTED ANSWERS

Questions you are likely to get

The framework and its mechanics

What exactly is Kaleido?

A .NET framework for building business services as discoverable queries and executable steps. It runs business handlers, tracks process progress, and returns next-step guidance. Applications use its HTTP surface; AI assistants use tools generated by its MCP adapter.

Basis: Slides 01-02, 06-09.

Is it just an API wrapper or a catalog?

No. The registry describes operations, but the Process runtime also loads state, validates requests and step availability, invokes business code, persists the outcome, and returns how to continue. The reusable execution mechanism is as important as discovery.

Basis: Slides 02, 05-07.

How does it know which step comes next?

It evaluates declared dependencies and availability against recorded progress, together with next direction returned by the business handler. A step can be required, several steps can be available, or information can be pending. The interface presents those possibilities; the runtime checks the process rules.

Basis: Slides 05, 07, 09.

What does the developer still have to write?

The C# step and query contracts, metadata, and business implementations. The developer connects business systems, registers the relevant assemblies, and configures the host, authorization, and production storage. Kaleido does not invent domain logic or automatically convert arbitrary APIs into steps.

Basis: Slides 02, 06, 13.

What is the difference between a query and a step?

A query reads information, such as matching provider locations. A step performs a business action, such as recording the selected provider for a request, within a process. The worked example deliberately separates searching, acting, and reading the next-step guidance.

Basis: Slides 02, 10.

Q&A / SUGGESTED ANSWERS

Questions you are likely to get

AI, platform fit, continuity, and readiness

What makes it AI-native without a built-in model?

The implemented MCP adapter generates tools and schemas from the same registry used by other consumers. It supports queries, step calls, guided lookups, information requests, and continuation. The language model stays outside Kaleido; the business handlers remain the place where actions execute.

Basis: Slides 08-09.

Does the AI choose the business path or approve requests?

The service returns what is required or available, and the agent is guided to offer available choices to the user. Business decisions and approvals belong to the application and its authorized decision-makers. Prompt instructions and confirmation hints are not proof that a client obtained human consent.

Basis: Slides 05, 09, 13.

Is this replacing our workflow or case platform?

That is not the proposed role. Kaleido includes a process runtime for business actions, but the surrounding platform can retain assignments, human work, and service-level management. The named platform examples describe intended integration fit, not packaged connectors or certified integrations.

Basis: Slide 12.

What happens when the chat ends or another person takes over?

The intended pattern is for the next authorized consumer to reload current process state using the process ID. Durable storage and case-level access control must be supplied by the deployment. A process ID is not permission, and the presentation does not prove every cross-channel or cross-processor path.

Basis: Slide 11.

Is it ready for 1.0, and what value has been proven?

The deck is a pre-1.0 preview, not release approval. Reviewed concerns about credential delegation, retries, completion, and client consent still need closure and validation. Reduced duplicated integration is the intended value; the sources do not establish measured cost savings or adoption results.

Basis: Slides 03, 13; cleanup-chatgpt.md, sections 5 and 15; cleanup-devin.md, KAL-REL-001.

A useful line to return to: "The interface asks for the action. The service runs it and explains what can happen next."